

DEPARTMENT OF MECHANICAL ENGINEERING

MECHATRONICS & IOT 24MEC51

EXERCISE NO: 13

GURUTHARSAN S	- 24MER011
HARISH KUMAR K K	- 24MER015
MADHANRAJ K	- 24MER028
SADAAGOPAAN T R	- 24MER052

MECHATRONICS & IoT

ASSIGNMENT REPORT – 13

TITLE:

Design and develop a GPS-Based Vehicle Tracking and Fleet Management System. The system enables real-time vehicle tracking and efficient fleet monitoring for transportation management

Done By:

GURTHARSAN S	- 24MER011
HARISH KUMAR K K	– 24MER015
MADHANRAJ K	– 24MER028
SADAAGOPAAN T R	– 24MER052

Project Overview:

The GPS-Based Vehicle Tracking and Fleet Management System is designed to provide real-time monitoring of vehicles using GPS technology. The system determines the vehicle's geographical location and transmits the information to a remote monitoring platform.

The proposed system uses a GPS module such as NEO-6M to obtain latitude, longitude, speed, date, and time. An ESP8266 NodeMCU acts as the main controller and processes the GPS information before transmitting it through a wireless network.

The fleet operator can monitor vehicle locations remotely and use the collected information for route management, speed monitoring, vehicle utilization, and transportation planning.

Problem Statement

Traditional vehicle and fleet management systems may depend heavily on manual communication and driver reporting. This makes it difficult to continuously monitor vehicle movement and efficiently manage multiple vehicles.

The major problems include:

- Lack of real-time vehicle location information.
 - Difficulty monitoring multiple vehicles.
 - Inefficient route management.
 - Difficulty detecting unauthorized vehicle movement.
 - Lack of continuous speed monitoring.
 - Delayed detection of abnormal vehicle movement.
 - Lack of historical route information.
- Increased transportation and fuel-management challenges.

Therefore, a GPS-based tracking system is required to provide continuous vehicle monitoring and improve fleet-management efficiency.

.

Proposed Solution

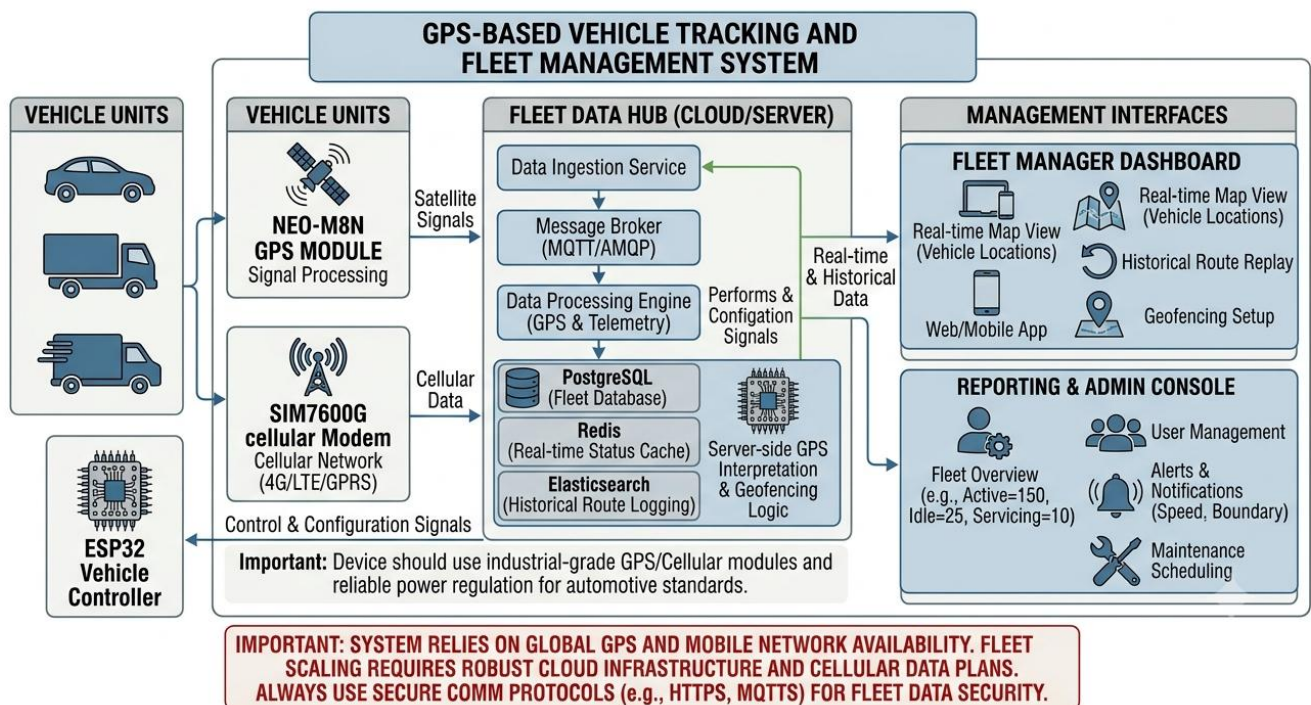
The proposed system consists of a GPS module, ESP8266 NodeMCU, wireless communication, cloud/server platform, and monitoring dashboard.

The GPS module receives signals from satellites and calculates the vehicle's location.

The ESP8266 receives and processes the GPS information. It can then transmit the data to a cloud server using an Internet connection.

The fleet operator can access the vehicle information remotely through a dashboard.

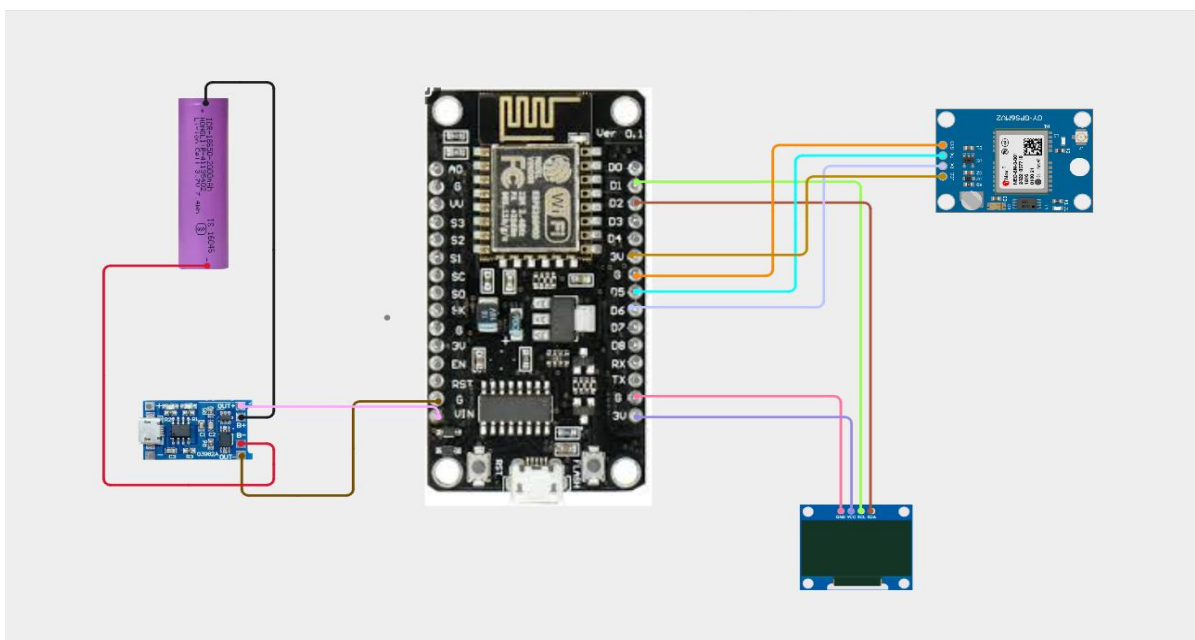
System Architecture



Circuit Table

S.No.	Component	Pin	ESP8266 Connection
1	NEO-6M GPS	VCC	Suitable supply
2	NEO-6M GPS	GND	GND
3	NEO-6M GPS	TX	D5 / GPIO14
4	NEO-6M GPS	RX	D6 / GPIO12
5	Green LED	Anode	D1 through 220 Ω
6	Green LED	Cathode	GND
7	Red LED	Anode	D2 through 220 Ω
8	Red LED	Cathode	GND
9	Buzzer	Positive	D7 / GPIO13
10	Buzzer	Negative	GND

Circuit Design



Algorithm

1. START
2. Initialize ESP8266 NodeMCU.
3. Initialize GPS module.
4. Initialize LEDs and buzzer.
5. Connect ESP8266 to the Internet.
6. Connect to the cloud/server.
7. Read GPS data.
8. Check whether valid GPS data is available.
9. If GPS data is NOT available:
 Display "GPS SEARCHING".
 Continue reading GPS data.
10. If valid GPS data is available:
 Read latitude.
 Read longitude.
 Read speed.
 Read date.
 Read time.
 Read satellite information.
11. Process the GPS information.
12. Determine vehicle status:
 If speed > 0:
 Status = MOVING.
 Else:
 Status = STOPPED.

13. Compare vehicle speed with the predefined speed limit.
14. If speed exceeds the limit:
Generate overspeed alert.
Turn ON red LED.
Turn ON buzzer.
15. Otherwise:
Keep normal status.
Turn ON green LED.
16. Send vehicle data to the cloud.
17. Update the fleet dashboard.
18. Store vehicle location data.
19. Wait for the next update interval.
20. Repeat the process continuously.
21. STOP.

Coding

```
// =====  
  
// GPS VEHICLE TRACKING AND FLEET MANAGEMENT SYSTEM  
  
// ESP8266 + NEO-6M + Adafruit IO  
  
// Upload every 20 seconds (Throttle Safe)  
  
// =====  
  
#include <ESP8266WiFi.h>  
  
#include "AdafruitIO_WiFi.h"
```

```
#include <TinyGPSPlus.h>
```

```
#include <SoftwareSerial.h>
```

```
// ----- WiFi -----
```

```
#define WIFI_SSID "Harish Kumar K K"
```

```
#define WIFI_PASS "Mavboiii@10"
```

```
// ----- Adafruit IO -----
```

```
#define IO_USERNAME "Jr_Frost"
```

```
#define IO_KEY "aio_obVD87ub3BbEG4osjIOQjHNrUn7m"
```

```
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);
```

```
// ----- Feeds -----
```

```
AdafruitIO_Feed *latitudeFeed = io.feed("latitude");
```

```
AdafruitIO_Feed *longitudeFeed = io.feed("longitude");
```

```
AdafruitIO_Feed *speedFeed = io.feed("speed");
```

```
AdafruitIO_Feed *satelliteFeed = io.feed("satellites");
```

```
AdafruitIO_Feed *locationFeed = io.feed("gps-location");
```

```
// ----- GPS Pins -----
```



```
// GPS TX -> D5 (GPIO14)
```

```
// GPS RX -> D6 (GPIO12)
```

```
SoftwareSerial gpsSerial(D5, D6);
```

```
TinyGPSPlus gps;
```

```
// Upload every 20 seconds
```

```
const unsigned long uploadInterval = 20000;
```

```
unsigned long lastUpload = 0;
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    gpsSerial.begin(9600);
```

```
    Serial.println();
```

```
    Serial.println("=====");
```

```
    Serial.println(" GPS VEHICLE TRACKING SYSTEM");
```

```
    Serial.println("=====");
```

```
    Serial.print("Connecting to Adafruit IO");
```

```
io.connect();
```

```
while (io.status() < AIO_CONNECTED) {
```

```
    Serial.print(".");
```

```
    delay(500);
```

```
}
```

```
Serial.println("\nConnected to Adafruit IO.");
```

```
Serial.println("Waiting for GPS Fix...");
```

```
}
```

```
void loop() {
```

```
    io.run();
```

```
    while (gpsSerial.available()) {
```

```
        gps.encode(gpsSerial.read());
```

```
    }
```

```
    if (!gps.location.isValid()) {
```

```
        static unsigned lo
```

```
ng lastMessage = 0;
```

```
if (millis() - lastMessage > 5000) {
```

```
    lastMessage = millis();
```

```
    Serial.println("Waiting for GPS satellites...");
```

```
}
```

```
return;
```

```
}
```

```
if (millis() - lastUpload >= uploadInterval) {
```

```
    lastUpload = millis();
```

```
    double latitude  = gps.location.lat();
```

```
    double longitude = gps.location.lng();
```

```
    double speed     = gps.speed.kmph();
```

```
    int satellites   = gps.satellites.value();
```

```
    double altitude  = gps.altitude.isValid() ? gps.altitude.meters() : 0;
```

```
    Serial.println("-----");
```

```
Serial.print("Latitude  : ");
```

```
Serial.println(latitude, 6);
```

```
Serial.print("Longitude  : ");
```

```
Serial.println(longitude, 6);
```

```
Serial.print("Speed    : ");
```

```
Serial.print(speed, 2);
```

```
Serial.println(" km/h");
```

```
Serial.print("Satellites : ");
```

```
Serial.println(satellites);
```

```
Serial.print("Altitude  : ");
```

```
Serial.print(altitude);
```

```
Serial.println(" m");
```

```
Serial.println("Uploading to Adafruit IO...");
```

```
latitudeFeed->save(latitude);
```

```
delay(200);
```

```
longitudeFeed->save(longitude);
```

```
delay(200);
```

```
speedFeed->save(speed);
```

```
delay(200);
```

```
satelliteFeed->save(satellites);
```

```
delay(200);
```

```
locationFeed->save(speed, latitude, longitude, altitude);
```

```
Serial.println("Upload Successful.");
```

```
Serial.println("-----");
```

```
}
```

```
}
```

System Working

Step 1 – GPS Data Collection

The GPS module receives signals from GPS satellites and determines the vehicle's geographical location.

The module provides latitude, longitude, speed, date, and time.

Step 2 – Data Processing

The ESP8266 receives the GPS data and processes it.

It determines whether the vehicle is:

- Moving
- Stopped
- Overspeeding
-

Step 3 – Internet Communication

The ESP8266 transmits the processed information to the cloud/server using an available Internet connection.

Step 4 – Cloud Storage

The cloud server stores the vehicle's location and movement information

.

Step 5 – Remote Monitoring

The fleet operator can access the dashboard remotely and monitor the vehicle.

Step 6 – Alert Generation

If the vehicle exceeds the predefined speed limit, the system can generate an overspeed alert.

Bills Of Material

S.No.	Component	Specification	Quantity	Purpose
1	ESP8266 NodeMCU	Wi-Fi Microcontroller	1	Main Controller
2	NEO-6M GPS Module	GPS Receiver	1	Location Tracking
3	Green LED	5 mm	1	Normal Indication
4	Red LED	5 mm	1	Alert Indication
5	Resistor	220 Ω	2	LED Protection
6	Buzzer	3.3/5 V	1	Overspeed Alert
7	Breadboard	Standard	1	Prototype Assembly
8	Jumper Wires	Male-Male	1 Set	Circuit Connections
9	USB Cable	Micro USB	1	Programming/Power
10	Power Supply	Suitable regulated supply	1	System Power

Prototype Implementation

The prototype consists of:

- ESP8266 NodeMCU
- NEO-6M GPS module
- LEDs
- Buzzer
- Breadboard
- Jumper wires
- Suitable power supply

The GPS module is connected to the ESP8266 using serial communication. After powering the system, the GPS module searches for satellites. Once a valid GPS fix is obtained, it provides location and speed information.

The ESP8266 processes the information and transmits it to a remote cloud/server platform through the Internet.

The fleet operator can then view the vehicle's current location and status through the dashboard.

Results & Performance Analysis

The system successfully demonstrates GPS-based vehicle tracking and basic fleet monitoring.

Parameter	Result
GPS Location Detection	Yes
Latitude Detection	Yes
Longitude Detection	Yes
Speed Monitoring	Yes
Vehicle Status Detection	Yes
Overspeed Detection	Yes
Wireless Communication	Supported
Cloud Monitoring	Supported
Remote Monitoring	Supported
Historical Tracking	Supported

Performance Analysis

The system enables fleet operators to remotely monitor vehicle movement and location. GPS data can be used to analyze routes and vehicle utilization.

The system can help with:

- Route planning.
- Vehicle scheduling.
- Driver monitoring.
- Trip analysis.
- Unauthorized route detection.
- Fleet utilization.
- Overspeed monitoring.

For wide-area real-time fleet tracking, a cellular communication module such as 4G/LTE is generally more practical than relying only on Wi-Fi.

